

1 LabLink: On-demand GPU teaching labs accessible 2 through the browser

3 **Andrew Park** ¹, **Elizabeth Berrigan** ¹, **Liezl Maree** ¹, and **Talmo**
4 **Pereira** ¹¶

5 ¹ Salk Institute for Biological Studies, La Jolla, California, United States ¶ Corresponding author

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright
and release the work under a
Creative Commons Attribution 4.0
International License ([CC BY 4.0](#)).

6 Summary

7 Quantitative biology increasingly relies on GPU-accelerated deep learning tools with graphical
8 interfaces, but teaching them hands-on is hard: installation consumes the opening hours
9 of a workshop, and many participants' laptops have no GPU. LabLink is an open-source
10 Infrastructure-as-Code platform with which a research group deploys reproducible GPU desktops
11 in its own Amazon Web Services (AWS) account or on any machines that run Docker. Its
12 allocator manages and monitors participant machines, whether provisioned on AWS or registered
13 by the operator. Each participant claims a seat by entering an email address on a web page and
14 lands in a full Linux desktop with the instructor's software preinstalled, running in a browser
15 tab ([Figure 1](#)). In our benchmark, a 30-seat pool was ready about seven minutes after a cold
16 start. LabLink has served approximately 600 participants across 15 events, from K–12 students
17 to faculty.

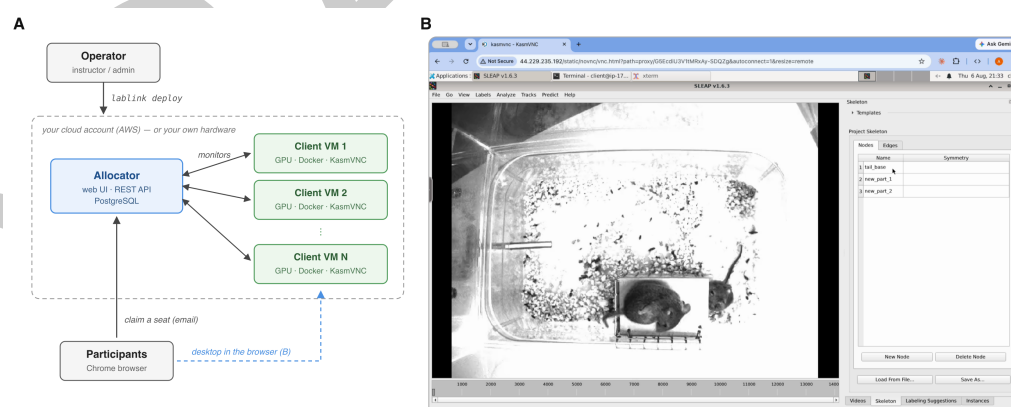


Figure 1: LabLink overview. (A) An operator deploys the allocator into their own cloud account or onto a lab-owned server with a single CLI command; the allocator manages one client VM per participant (provisioned on AWS, or registered bring-your-own machines) and monitors their health. Participants enter an email address and receive a full Linux desktop streamed to their browser. (B) The participant's view: SLEAP v1.6.3 running in a LabLink browser desktop.

18 Statement of need

19 Hands-on workshops are how research software spreads ([Wilson, 2016](#)), but GPU-dependent
20 scientific software is hard to teach. Installation means resolving dependency stacks, GPU
21 drivers, and operating-system-specific configuration ([Mangul et al., 2019](#)), and the first hours

of a workshop are routinely lost to it. Many participants also have no local GPU, so even a successful installation cannot run training or inference.

The common workarounds each sacrifice something: asking participants to preinstall shifts the burden onto the least experienced users, shared teaching servers require institutional infrastructure most labs lack, and cloud notebooks solve hardware access but do not, by default, run desktop applications with graphical interfaces. LabLink closes this gap: a lab creates a pool of identical, isolated, GPU-backed Linux desktops in its own cloud account or on machines it already has, hands each participant one link, and tears everything down when the workshop ends. It targets maintainers and instructors of GPU-dependent scientific software who must equip a room of participants without per-laptop setup or institutional computing infrastructure.

State of the field

Domain platforms such as brainlife.io (Hayashi et al., 2024), Neurodesk (Renton et al., 2024), and brainchop (Masoud et al., 2023) deliver cloud-based or in-browser neuroimaging environments, and NeuroCAAS (Abe et al., 2022) applies infrastructure-as-code to neuroscience analyses run as batch jobs. Each serves its own application catalog, not software the instructor packages.

General-purpose teaching infrastructure is notebook-centric. JupyterHub (Project Jupyter contributors, 2026a) can stream a Linux desktop through jupyter-remote-desktop-proxy (Project Jupyter contributors, 2026b), but as a standing service someone must keep running: on one server with The Littlest JupyterHub (Project Jupyter contributors, 2026c), on a Kubernetes cluster for larger classes and for BinderHub (Project Jupyter et al., 2018), or hosted by a provider such as 2i2c (2i2c, 2026). LabLink starts one level down: from an image of the instructor's software and the lab's cloud credentials, one command provisions a GPU machine per participant and a second removes all of it, so the lab operates nothing between events. Browser development environments such as GitHub Codespaces (GitHub, Inc., 2026), Gitpod (Gitpod GmbH, 2026), and Coder (Coder Technologies, Inc., 2026) offer editors and terminals rather than GPU desktops, and Google Colab (Google LLC, 2026b) offers GPUs only in notebooks, with session limits and no instructor control. The Galaxy project's Training Infrastructure as a Service (Rasche et al., 2023) serves Galaxy workflows on shared public servers, not an instructor's own software.

Browser remote desktops are well established: Apache Guacamole (The Apache Software Foundation, 2026) gateways a browser to existing machines, Open OnDemand (Hudak et al., 2018) delivers desktops from an institutional HPC cluster, and Kasm Workspaces (Kasm Technologies, 2026a) orchestrates containerized desktops as an account-based service an administrator maintains. Cloud vendors do the same: Azure Lab Services (Microsoft, 2026) provisioned per-student VM pools but retires in June 2027, and Amazon WorkSpaces (Amazon Web Services, 2026b) streams managed GPU desktops bound to AWS.

LabLink's contribution is the integration around the desktop rather than the desktop itself: one GPU machine per participant, provisioned reproducibly in infrastructure the operator owns, running instructor-packaged software, with countless email-based seat claiming, health monitoring with automatic recovery, and teardown when the event ends. We wrote new software because the missing piece is the workshop lifecycle: existing platforms are standing services an administrator maintains, whereas a LabLink deployment is created for one event and destroyed afterward.

Software design

LabLink has two components, split by where the work has to happen. A single **allocator** per deployment holds the cloud credentials that run OpenTofu, the database of machines and seats,

the participant-facing web page, and the proxy through which desktops are reached. A client service runs on every machine and does the work that must happen on that machine: it registers the machine, reports heartbeats, GPU health, and seat status, ships its logs, and rotates the desktop password when a seat is claimed. Because every report is a client-initiated REST call, a machine needs no public address and no inbound rule from the internet, which lets machines LabLink did not provision, even behind an institutional firewall, join the same pool. On AWS, and in every bring-your-own topology except direct LAN access, the participant's desktop is streamed over WebSockets and proxied through the allocator's nginx (nginx contributors, 2026), so the allocator is the deployment's single participant-facing endpoint.

The allocator is a Flask (Pallets, 2026) service backed by PostgreSQL (The PostgreSQL Global Development Group, 2026), chosen over an embedded database because row-level locking lets a burst of simultaneous claims each take a different free machine. Cloud machines are provisioned by OpenTofu (OpenTofu contributors, 2026) runs in the background; declarative infrastructure makes teardown of a per-event deployment's compute repeatable, and only a small state bucket and lock table persist between events. It also serves the admin dashboard and can destroy the pool at a scheduled time. On AWS it sweeps the pool once a minute and reboots any machine that reports a startup failure, fails its GPU check, stalls while initializing, or goes silent: cold if unassigned, and warm, preserving the session, if assigned; after three failures its seat is released. Bring-your-own hosts are monitored but not rebooted.

Each client machine runs one Docker (Merkel, 2014) container built from the instructor's image, hosting both the KasmVNC (Kasm Technologies, 2026b) desktop and the client service. Giving each participant a whole machine rather than a slice of a shared server trades cost for isolation: one participant's crashed session or saturated GPU affects only their own seat. At us-west-2 on-demand prices (Amazon Web Services, 2026a), a g4dn.xlarge seat costs about \$0.53 per hour, roughly \$2 per participant for a four-hour workshop.

KasmVNC replaced Chrome Remote Desktop (Google LLC, 2026a), used for LabLink's first events, which required every participant to hold a Google account and, in our experience, could not be version-pinned: two successive releases of its apt package broke the client image, and superseded versions left Google's repository within weeks. KasmVNC is open source, needs only a browser, and is pinned to an exact version in the client image.

The allocator's own infrastructure is defined in a separate template repository, deployed either from a laptop with lablink-cli or, for reviewed infrastructure changes, by forking the template and using its GitHub Actions workflows; both deploy the same allocator image. The manual provider is how bring-your-own machines join: the allocator registers, monitors, and proxies them but leaves their creation and teardown to the operator, so only the AWS provider delivers the full provision-to-destroy lifecycle.

Development and release

Each package is tested and published to PyPI independently (lablink-allocator-service, lablink-client-service, lablink-cli), with 90% coverage enforced on the allocator and client and the allocator's OpenTofu tests run against AWS in CI. The image a workshop pins is built from the same artifact anyone can install. Working-tree builds are tagged -test with their commit hash and tested inside the image; production images are built only from published PyPI versions, so any production image can be rebuilt from its version number. Documentation is at <https://lablink.talmolab.org/>.

Security

TLS is a configuration option, not a default; a bare-IP deployment runs over plain HTTP. Operator pages require operator credentials; client machines hold only per-machine secrets minted at registration. On AWS a client machine accepts inbound connections only from

the allocator and from operator SSH, and participants are administrators inside their desktop container but not on the host.

The seat-claim page is deliberately unauthenticated: an email address labels a seat rather than authenticating a user, and nothing gates it. This is acceptable only because a deployment is short-lived with a fixed pool: a leaked link can exhaust seats but cannot provision machines, one address holds one seat, and scheduled destruction bounds the exposure. Claiming a seat rotates the desktop password and binds the browser to a signed session cookie, so participants cannot open one another's seats. The address is never verified and dies with the deployment.

Configuration and adaptability

One YAML file describes a deployment and is read by both deployment paths and the allocator; apart from the startup script there are deliberately no per-setting overrides, so the file is the version-controllable record of a deployment. It answers three questions: (1) where machines come from; (2) how the allocator reaches each desktop and participants reach the allocator (on AWS, the private network; for bring-your-own machines, the LAN, a WireGuard mesh overlay, or a client-initiated WebSocket tunnel, plus a Tailscale Funnel ([Tailscale Inc., 2026](#)) or Cloudflare Tunnel ([Cloudflare, Inc., 2026](#)) when the allocator has no public address); and (3) what runs on each desktop. The last is a startup script that installs the instructor's software as the container boots, with retries and an optional success check, since parallel installs fail intermittently at scale. Thus, adapting to new software means editing these two files.

Benchmark

We deployed pools of 5, 10, 30, 60, and 150 g4dn.xlarge seats in us-west-2, one run per pool size ([Figure 2](#)). Per-VM readiness runs from the pool-launch command until the allocator marks the VM assignable, excluding the one-time allocator deploy; statistics are medians and interquartile ranges across a run's VMs. Script, metadata, and raw data are in `paper/fig2-harness/`.

From a cold start, a ready 30-seat pool takes about seven minutes. At 5 to 60 seats the per-client median readiness time stayed near five minutes. The 150-seat run, the size of the largest event in [Table 1](#), had all VMs ready 11 minutes after the launch command with a per-client median of about six and a half minutes; its infrastructure apply took 249 s against 83 s at 30 seats, while per-client boot time grew only modestly. All 255 VMs across the five pools reached ready ([Figure 2C](#), with counts annotated above each bar). Ten (two at 10 seats, eight at 150) needed one automatic reboot after reporting a startup failure at the same point during the client-image pull, pointing to a transient registry failure rather than faulty machines; all ten were ready after the second boot.

Table 1: Workshops and courses delivered on LabLink, 2024–2026. Participant counts marked ~ are organizer estimates.

Event	Date(s)	Participants	Location
Cosyne Main Tutorial – SLEAP	Feb 29, 2024	~150	Lisbon, Portugal
University of Illinois Urbana–Champaign workshop	Apr 2, 2024	~40	Urbana, IL, USA
Keiller Leadership Academy	May 30, 2024	~125 (K–12 students)	San Diego, CA, USA
STEM Day	May 30, 2024	~125 (K–12 students)	San Diego, CA, USA
CAJAL Advanced Neuroscience Training Programme	Jun 2024	~20	Champalimaud Centre for the Unknown, Lisbon, Portugal
Salk EDGE Summer Program	Jul 24, 2024	3	La Jolla, CA, USA
NYU Langone Neuroscience T32 Workshop	Aug 19, 2024	~25	New York, NY, USA
Ellen Potter Research Connections	Apr 9–10, 2025	~20	Salk Institute, La Jolla, CA, USA
Bridges to the Future Symposium	May 28, 2025	~50	Salk Institute, La Jolla, CA, USA
CAJAL Advanced Neuroscience Training Programme	Jun 8–27, 2025	~30	Champalimaud Centre for the Unknown, Lisbon, Portugal
Zagreb NeuroData Summer School	Jul 2025	~30	Online / Zagreb, Croatia
Salk EDGE Summer Program	Jul 14–15, 2025	2	Salk Institute, La Jolla, CA, USA
Computer Vision for Biologists	Sep 1–12, 2025	~30	University of California Davis, CA, USA
Introduction to Multi-Animal Pose Tracking (US-RSE 2025)	Oct 6, 2025	~30	Philadelphia, PA, USA
Short course on machine learning applications	Oct 13–17, 2025	~30	Jackson Laboratory, Bar Harbor, ME, USA
CAJAL Advanced Neuroscience Training Programme	Jun 10, 2026	18	Champalimaud Centre for the Unknown, Lisbon, Portugal

AI usage disclosure

Claude Opus 5 and Claude Fable 5.1 (Anthropic), through Claude Code, assisted with both the software and this manuscript. The human authors designed and wrote the architecture and initial implementation; Claude Code later drafted code and documentation, which the authors reviewed and validated with the automated test suites and in live deployments. The authors drafted the paper; Claude reviewed and shortened the text, checked citations, and helped assemble the benchmark figure, which the committed script regenerates from recorded run data. All text and figures were reviewed by the authors.

Acknowledgements

This work was supported by the NIH/BRAIN Initiative RF1 award under grant number 1R1MH132653-01. We thank the open-source projects that made this work possible, including OpenTofu, Docker, KasmVNC, and PostgreSQL.

References

- 2i2c. (2026). *2i2c: Managed JupyterHub infrastructure for research and education*. <https://2i2c.org/>
- Abe, T., Kinsella, I., Saxena, S., Buchanan, E. K., Couto, J., Briggs, J., Kitt, S. L., Glassman, R., Zhou, J., Paninski, L., & Cunningham, J. P. (2022). Neuroscience Cloud Analysis As a Service: An open-source platform for scalable, reproducible data analysis. *Neuron*, 110(17), 2771–2789.e7. <https://doi.org/10.1016/j.neuron.2022.06.018>
- Amazon Web Services. (2026a). *Amazon EC2 on-demand pricing*. <https://aws.amazon.com/ec2/pricing/on-demand/>
- Amazon Web Services. (2026b). *Amazon WorkSpaces and Amazon WorkSpaces Applications (formerly AppStream 2.0)*. <https://aws.amazon.com/workspaces/>
- Cloudflare, Inc. (2026). *Cloudflare Tunnel*. <https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/>
- Coder Technologies, Inc. (2026). *Coder*. <https://coder.com>
- GitHub, Inc. (2026). *GitHub Codespaces*. <https://github.com/features/codespaces>
- Gitpod GmbH. (2026). *Gitpod*. <https://www.gitpod.io>
- Google LLC. (2026a). *Chrome Remote Desktop*. <https://remotedesktop.google.com>
- Google LLC. (2026b). *Google Colaboratory*. <https://colab.research.google.com>
- Hayashi, S., Caron, B. A., Heinsfeld, A. S., Vinci-Booher, S., McPherson, B., Bullock, D. N., Bertò, G., Niso, G., Hanekamp, S., Levitas, D., Ray, K., MacKenzie, A., Avesani, P., Kitchell, L., Leong, J. K., Nascimento-Silva, F., Koudoro, S., Willis, H., Jolly, J. K., ... Pestilli, F. (2024). brainlife.io: A decentralized and open-source cloud platform to support neuroscience research. *Nature Methods*, 21, 809–813. <https://doi.org/10.1038/s41592-024-02237-2>
- Hudak, D., Johnson, D., Chalker, A., Nicklas, J., Franz, E., Dockendorf, T., & McMichael, B. L. (2018). Open OnDemand: A web-based client portal for HPC centers. *Journal of Open Source Software*, 3(25), 622. <https://doi.org/10.21105/joss.00622>
- Kasm Technologies. (2026a). *Kasm Workspaces*. <https://kasm.com>
- Kasm Technologies. (2026b). *KasmVNC: Modern VNC server and client, web based and secure*. <https://github.com/kasmtech/KasmVNC>
- Mangul, S., Mosqueiro, T., Abdill, R. J., Duong, D., Mitchell, K., Sarwal, V., Hill, B., Brito, J., Littman, R. J., Statz, B., Lam, A. K.-M., Dayama, G., Grieneisen, L., Martin, L. S., Flint, J., Eskin, E., & Blekhman, R. (2019). Challenges and recommendations to improve the installability and archival stability of omics computational tools. *PLOS Biology*, 17(6), e3000333. <https://doi.org/10.1371/journal.pbio.3000333>
- Masoud, M., Hu, F., & Plis, S. (2023). Brainchop: In-browser MRI volumetric segmentation and rendering. *Journal of Open Source Software*, 8(83), 5098. <https://doi.org/10.21105/joss.05098>
- Mathis, A., Mamidanna, P., Cury, K. M., Abe, T., Murthy, V. N., Mathis, M. W., & Bethge, M. (2018). DeepLabCut: Markerless pose estimation of user-defined body parts with deep

- learning. *Nature Neuroscience*, 21(9), 1281–1289. <https://doi.org/10.1038/s41593-018-0209-y>
- Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal*, 2014(239), 2.
- Microsoft. (2026). *Azure Lab Services retirement guide*. <https://learn.microsoft.com/en-us/azure/lab-services/retirement-guide>
- nginx contributors. (2026). *nginx*. <https://nginx.org>
- OpenTofu contributors. (2026). *OpenTofu: The open source infrastructure as code tool*. <https://opentofu.org>
- Pachitariu, M., Sridhar, S., Pennington, J., & Stringer, C. (2024). Spike sorting with Kilosort4. *Nature Methods*, 21(5), 914–921. <https://doi.org/10.1038/s41592-024-02232-7>
- Pallets. (2026). *Flask*. <https://flask.palletsprojects.com>
- Pereira, T. D., Tabris, N., Matsliah, A., Turner, D. M., Li, J., Ravindranath, S., Papadoyannis, E. S., Normand, E., Deutsch, D. S., Wang, Z. Y., McKenzie-Smith, G. C., Mitelut, C. C., Castro, M. D., D'Uva, J., Kislin, M., Sanes, D. H., Kocher, S. D., Wang, S. S.-H., Falkner, A. L., ... Murthy, M. (2022). SLEAP: A deep learning system for multi-animal pose tracking. *Nature Methods*, 19(4), 486–495. <https://doi.org/10.1038/s41592-022-01426-1>
- Project Jupyter contributors. (2026a). *JupyterHub: Multi-user server for Jupyter notebooks*. <https://github.com/jupyterhub/jupyterhub>
- Project Jupyter contributors. (2026b). *jupyter-remote-desktop-proxy: Run a Linux desktop on a JupyterHub*. <https://github.com/jupyterhub/jupyter-remote-desktop-proxy>
- Project Jupyter contributors. (2026c). *The Littlest JupyterHub*. <https://github.com/jupyterhub/the-littlest-jupyterhub>
- Project Jupyter, Bussonnier, M., Forde, J., Freeman, J., Granger, B., Head, T., Holdgraf, C., Kelley, K., Nalvarte, G., Osherooff, A., Pacer, M., Panda, Y., Pérez, F., Ragan-Kelley, B., & Willing, C. (2018). Binder 2.0 – reproducible, interactive, sharable environments for science at scale. *Proceedings of the 17th Python in Science Conference*, 113–120. <https://doi.org/10.25080/Majora-4af1f417-011>
- Rasche, H., Hyde, C., Davis, J., Gladman, S., Coraor, N., Bretaudeau, A., Cuccuru, G., Bacon, W., Serrano-Solano, B., Hillman-Jackson, J., Hiltmann, S., Zhou, M., Grüning, B., & Stubbs, A. (2023). Training Infrastructure as a Service. *GigaScience*, 12, giad048. <https://doi.org/10.1093/gigascience/giad048>
- Renton, A. I., Dao, T. T., Johnstone, T., Civier, O., Sullivan, R. P., White, D. J., Lyons, P., Slade, B. M., Abbott, D. F., Amos, T. J., Bollmann, S., Botting, A., Campbell, M. E. J., Chang, J., Close, T. G., Dörig, M., Eckstein, K., Egan, G. F., Evas, S., ... Bollmann, S. (2024). Neurodesk: An accessible, flexible and portable data analysis environment for reproducible neuroimaging. *Nature Methods*, 21, 804–808. <https://doi.org/10.1038/s41592-023-02145-x>
- Romero-Ferrero, F., Bergomi, M. G., Hinz, R. C., Heras, F. J. H., & Polavieja, G. G. de. (2019). idtracker.ai: Tracking all individuals in small or large collectives of unmarked animals. *Nature Methods*, 16(2), 179–182. <https://doi.org/10.1038/s41592-018-0295-5>
- Stringer, C., Wang, T., Michaelos, M., & Pachitariu, M. (2021). Cellpose: A generalist algorithm for cellular segmentation. *Nature Methods*, 18(1), 100–106. <https://doi.org/10.1038/s41592-020-01018-x>
- Tailscale Inc. (2026). *Tailscale Funnel*. <https://tailscale.com/kb/1223/funnel>
- The Apache Software Foundation. (2026). *Apache Guacamole*. <https://guacamole.apache.org>

- 264 The PostgreSQL Global Development Group. (2026). *PostgreSQL*. <https://www.postgresql.org>
- 265 Wilson, G. (2016). Software Carpentry: Lessons learned. *F1000Research*, 3, 62. [https:](https://doi.org/10.12688/f1000research.3-62.v2)
- 266 [//doi.org/10.12688/f1000research.3-62.v2](https://doi.org/10.12688/f1000research.3-62.v2)

DRAFT